

From Nand to Tetris

NPU Extension

Chapter 2: 脉动阵列

PE and Systolic Array

Project

Building a Modern Computer From First Principles

Background

上一章确定了 NPU 的系统架构。本章开始写真正的 RTL：先是单个乘累加单元 PE，再把它排列成 $N \times N$ 的 weight-stationary 脉动阵列。

仓库里已经搭好作业框架：`solution/06/` 有完整答案，`assignment/06/` 有留空模板，`tb/06/` 有测试台。你的任务是补全模板并通过 `make sim-06`。

Objective

实现 `n2t\_pe`：int8 x int8 + int32 累加的 MAC 单元。

实现 `n2t\_systolic\_array`：参数化 $N \times N$ 脉动阵列，支持批量权重装载、斜输入和输出对齐。

理解阵列语义： $C[k][col] = \sum_{row} A[row][k] * W[row][col]$ 。

Deliverables

- `assignment/06/PE.v`：完成后的 PE。
- `assignment/06/SystolicArray.v`：完成后的阵列。
- 运行 `make sim-06` 得到全部 PASS。

Contract

- PE： $psum_out = psum_in + a_in * w$ ，全部使用有符号运算。
- 阵列：`w\_data` 为 $N*N*W_W$ 位，`a\_data` 为 $N*A_W$ 位，`psum\_out` 为 $N*P_W$ 位。
- 时序：统一在 clk 下降沿提交，与仓库其他模块一致。
- 测试：PE 6 项检查、SystolicArray 64 项检查全部通过。

Resources

- `docs/npu.md`：数据布局、时序约定。
- `solution/06/PE.v` 与 `solution/06/SystolicArray.v`：参考实现。
- `tb/06/PE\_tb.v` 与 `tb/06/SystolicArray\_tb.v`：测试台。

Tips

- 先做 PE，用单独测试台验证乘法与累加，再做阵列。
- 注意 Verilog 有符号：声明 `signed` 并用 32 位累加，防止截断。
- 斜输入由阵列内部延迟实现；输出对齐消除列间偏移。
- 调试时先用 4x4 参数跑通，再回到 8x8。